COMP 5567 Distributed Algorithms and Protocols for Blockchains 2026 Fall HotStuff Project Specification

Objective and Requirements

Complete the missing core functions in the supplied starter code. Task A implements the **HotStuff consensus protocol.** Task B implements a **blockchain transaction system** on top of it. Demonstrate your implementation and explain how consensus commitment triggers deterministic execution.

Work in groups of at most **five**. All groups complete the basic requirements of Tasks A and B. Optional extensions earn the remaining task marks within the 100-point total. Record each member's contribution and identify incomplete features.

Teaching repository: https://github.com/goudanwang/hotstuff

It contains TODO functions, interface contracts and public tests. Implement the TODOs yourself; unchanged solution code copied from the reference repository does not count as your implementation.

Schedule

Activity	Arrangement
Presentation and demonstration	29 November 2026, 18:30–21:30, Z504
Report and project submission	Before 2 December 2026, 23:59, via Canvas
Each group	4 minutes slides + 4 minutes live operation + 2 minutes Q&A

Grading

Component	Maximum
Task A HotStuff consensus protocol implementation	35
Task B HotStuff blockchain system implementation	30
Report and presentation	20
Live demonstration	10
Individual contribution	5
Total	100

Task A has 30 basic marks and 5 extension marks. Task B has 25 basic marks and 5 extension marks. Completing only the basic features can earn at most 90/100 overall. Extensions are optional and do not raise the total above 100.

Framework

Python, FastAPI and the browser dashboard are provided. One process simulates four replicas with a quorum of three. Keep the existing interfaces, controls and observable state. The event queue, initial data, API routes and display remain available; consensus and transaction algorithms are left as TODOs.

Real networking, threshold signatures and disk persistence are not required. Basic QCs use simulator-authenticated node identities. Public tests illustrate requirements; grading may include additional inputs under the same contracts.

Task A HotStuff Consensus Protocol Implementation

35 marks = 30 basic + 2 recovery + 3 malicious-node handling. Implement the protocol using empty blocks first, so Task A can be tested independently of Task B.

Item	Marks	Required behavior
A1 Proposal	6	Complete propose(): select the parent using HighQC, then construct and distribute a proposal for the current leader and view.
A2 Safe voting	8	Complete vote(), Node.extends() and Node.can_vote(): validate the leader, view and parent QC; vote at most once per view; enforce the locked-branch or newer-QC rule.
A3 Quorum certificate	6	Complete form_qc(): count distinct valid voters and form a QC only with at least three votes for the same block and view. Deliver it through dispatch_qc().
A4 Lock and commit	8	Complete Node.receive_qc(): update HighQC and LockedQC; commit a directly linked certified three-chain in consecutive views, including uncommitted ancestors in order. One QC alone must not commit its own block.
A5 Normal view change	2	Complete next_view() and Node.enter_view(): advance the view and rotate the leader after a successful round while preserving protocol state.
A6 Recovery OPTIONAL	2	Complete timeout(): change leader after a crash and resume with three live replicas (1). Complete recover(): catch up a crashed replica and preserve voting history (1). Never lower the quorum; two available voters must stall. In-memory recovery is sufficient.
A7 Malicious nodes OPTIONAL	3	Complete inject_attack() and extend vote(), form_qc() or Node.receive_qc() as needed. Handle conflicting proposals in one view (2), plus duplicate votes or a wrong-view/invalid QC (1). Reject invalid evidence and prevent conflicting commits. Vote withholding alone earns no A7 marks.

Implementation Setting

Unqualified functions above belong to HotStuffDemo in backend/hotstuff.py; Node functions are in backend/node.py. Task A determines committed block IDs. The provided dispatch_qc() helper connects commitment to Task B execution. Keep transaction execution out of the voting and lock rules. A7 uses trusted simulator identities; cryptographic vote signatures are not required.

Task B HotStuff Blockchain System Implementation

30 marks = 25 basic + 2 batch configuration + 3 transaction security. Build on Task A and use the supplied accounts Alice, Bob and Charlie, each initially holding 100 units.

Item	Marks	Required behavior
B1 Admission and pool	5	Complete submit(): validate accounts and integer amounts, assign transaction IDs and maintain the pending transaction pool. Admission must not change balances.
B2 Block payload	5	Complete select_transactions(): select up to five transactions by default and exclude IDs on the chosen parent chain. Retain uncommitted transactions, including those in abandoned proposals. Allow empty blocks.
B3 State execution	8	Complete Node.execute_committed() and the execution logic in StateMachine.apply(): execute only committed transactions in block and payload order. Check funds; reject insufficient balances without balance changes. Keep correct replicas consistent.
B4 Receipts and retries	7	Complete the receipt and deduplication logic in StateMachine.apply(), and finalize_transactions(): record success or failure, prevent repeat execution of the same ID and clean committed transactions from the pool. Failed executions also receive final receipts.
B5 Batch configuration OPTIONAL	2	Complete configure_batch() and update select_transactions(): expose a positive-integer limit through the API or UI, support limits 1, 2 and 5, and reject invalid settings.
B6 Transaction security OPTIONAL	3	Add sign_transaction(), verify_transaction() and transaction_id(); extend submit() and StateMachine.apply(). Verify sender signatures before admission and at execution (2). Enforce nonces and stable IDs so repeated signed requests cannot execute twice (1).

Unqualified existing functions above are in backend/hotstuff.py; Node is in backend/node.py; StateMachine is in backend/blockchain.py. Add the B6 helpers in a module of your choice and extend the transaction/request fields as needed.

Implementation Setting

Use a **standard signature library and classroom test keys**. Sign a canonical payload containing chain/domain, sender, receiver, amount and nonce. Verify before admission and again at execution. Reject tampering and signatures from the wrong account.

Each account starts at nonce 0. Only committed execution consumes a matching nonce, once, including an authenticated insufficient-funds result. Invalid signatures and wrong nonces consume nothing. Identical retries return the same pending identity or final receipt. Derive identity from the payload, not its signature; document your policy for future nonces. If recovery is implemented, preserve nonce and receipt state.

Report and Presentation

20 marks. Submit a report of at most six main-text pages; references and the contribution table are excluded. Use four sections: design and implementation, Task A and Task B mechanisms, tests and results, and limitations.

Component	Marks	Assessment
Technical report	10	Implementation and key functions (4); tests and results (4); limitations and discussion (2).
PPT	4	A clear sequence: system design, Task A, Task B, test results and limitations.
Presentation	3	Accurate, understandable explanation with coordinated delivery within four minutes.
Q&A	3	Explain the submitted code, protocol mechanisms and observed results; every member must be available.

Live Demonstration

10 marks in four minutes. Operate the running system in real time. Show Task A proposal, votes, QC and commitment (5), then Task B transaction submission, balances before and after commitment, receipts and one failing case (5). Prepare the system before your slot.

Replacing the live demonstration with a prerecorded video receives 0/10 for this component. Screenshots and recordings do not earn live-operation marks.

Optional features are assessed within Tasks A and B. They are not required for full live-demonstration marks. Keep the two-minute Q&A separate from the four-minute live demonstration.

Individual Contribution

5 marks per student: a clear description of personal modules, functions, tests or analysis (2), supported by verifiable code, commits or other work products (3). Include one group contribution table with names, student IDs, responsibilities and evidence. Q&A marks assess answers; contribution marks assess the work performed.

Submission

Upload one COMP5567_HotStuff_GroupXX.zip archive to Canvas containing:

- source/ with the completed code and dependency file; tests/ and results/ with reproducible cases and outputs.
- README.md with installation, launch, test commands, the starter/base version and completed extensions.
- report.pdf, presentation.pdf or presentation.pptx, and the contribution table.
- changes.md or a patch identifying modified functions and known incomplete requirements.

Do not submit virtual environments, dependency folders or a demonstration video. Retain public-test intent when adapting interfaces; explain changes and add tests for claimed extensions.

Warning and References

AI Use Policy

AI may be used only to **assist the implementation or debugging of individual functions.** Students must **not** use AI agents to implement the entire project or a complete project component.

All AI-assisted code **must be clearly marked**, and the report must state **which functions used AI and what assistance was provided**.

Penalties:

- Entire project/component completed by AI: 0 marks for that project/component.
- AI-assisted code not disclosed: deduction of the corresponding marks.
- Student cannot explain an AI-assisted function in the report/assessment: deduction up to all marks for that function.

All reports will undergo the University's AI-detection checks; please ensure that your work complies with PolyU's academic integrity requirements.

[1] Teaching repository and baseline README: https://github.com/goudanwang/hotstuff

[2] M. Yin et al. HotStuff: BFT Consensus with Linearity and Responsiveness. PODC 2019. https://arxiv.org/abs/1803.05069

[3] PolyU academic integrity guidance: https://www.polyu.edu.hk/ogur/docdrive/Academic_Integrity/Plagiarism_Booklet.pdf